

NCAS - Nucleo Cognitivo da Aurora Siger

Documentação da lógica e requisitos do projeto

O NCAS é um protótipo em Python para registrar, organizar, consultar e interpretar dados operacionais de uma colônia espacial fictícia. O sistema combina arquivos TXT, JSON, regras de lógica booleana, engenharia de prompts e um modelo de linguagem local executado pelo Ollama.

1. Arquitetura do projeto

JSON: armazena os módulos e seus dados estruturados. TXT: mantém registros/histórico das operações. Python: controla o menu, cadastro, leitura e escrita dos arquivos e as regras de validação. Ollama + Qwen3:4B: interpreta os dados de módulos que apresentam risco e produz uma resposta estruturada.

2. Fluxo principal

Cadastro -> atualização do dicionário em memória -> gravação no JSON -> validação lógica -> identificação de risco -> consulta ao modelo local -> resposta estruturada -> registro no TXT.

3. Regra lógica utilizada

A regra operacional do NCAS é: $RISCO = F \vee A \vee I$, onde F representa falha crítica, A representa status de alerta e I representa integridade abaixo de 80%. Se qualquer uma das condições for verdadeira, o módulo é considerado de risco.

Variável	Significado	Condição
F	Falha crítica	<code>falha_critica == True</code>
A	Status de alerta	<code>status == "alerta"</code>
I	Integridade baixa	<code>integridade_percentual < 80%</code>

4. Simplificacao booleana

Para demonstrar uma simplificação equivalente à regra implementada, podemos expandir F usando a lei do terceiro excluído. Considere C como a condição 'módulo crítico':

$$R = (F \wedge A \wedge C) \vee (F \wedge A \wedge \neg C) \vee A \vee I$$

$$R = F \wedge A \wedge (C \vee \neg C) \vee A \vee I \text{ (fatoração/distributividade)}$$

$R = F \vee A \vee I$ (Cv -C = True)

$R = F \vee A \vee I$ (identidade)

A expressão final é exatamente a regra usada pelo Python: `risco = falha_critica or status_alerta or integridade_baixa`. A simplificação, portanto, não altera o comportamento do sistema.

5. Arquivos e persistência

O projeto utiliza `open()` com `with` para garantir o fechamento dos arquivos. O JSON é lido com `json.load()` e salvo com `json.dump()`. O TXT é utilizado para acrescentar registros por meio do modo `a`. O cadastro adiciona um novo dicionário à lista de módulos e persiste a alteração no arquivo JSON.

6. Prompts e IA

O projeto contempla três estratégias de prompt: Zero-Shot, quando a tarefa é apresentada sem exemplos; Few-Shot, quando um exemplo orienta o formato/comportamento esperado; e Structured Output, quando a resposta é solicitada em um formato JSON definido. O sistema também envia ao modelo o resultado da validação lógica para manter a decisão de risco sob controle do programa.

7. Modelo local e segurança

O assistente utiliza o Ollama localmente com o modelo Qwen3:4B. Assim, não é necessária uma chave de API externa para a execução. O Python prepara o contexto e recebe uma resposta estruturada do modelo. O sistema não solicita nem exibe raciocínio interno do modelo; a resposta utilizada pelo programa é objetiva e orientada à operação.

8. Conceitos contemplados

O projeto relaciona os conceitos estudados de manipulação de TXT, JSON e dicionários, álgebra booleana, LLMs, tokens, pretreinamento e previsão do próximo token, engenharia de prompts, memória/armazenamento/barramentos e responsabilidade no uso de IA. Também permite discutir otimização de modelos, regularização, MSE e descida do gradiente em nível conceitual.